Nuance Protocol: Master Build Plan

Production Hardening, Intelligent Contract Security, Fund Rescue Architecture & Full Stack Release

DOCUMENT TYPE	TARGET NETWORK	TARGET RELEASE	STATUS
Engineering Master Build Plan (PRD Appendix)	**GenLayer Testnet Bradbury (Chain 4221) → Mainnet**	**v1.0 Production Readiness**	**Approved for Implementation**

1. Executive Summary & Problem Diagnosis

Nuance settles qualitative human contracts through AI-validator consensus on GenLayer. During live testnet validation on Bradbury, a critical fund safety challenge emerged: users could deposit and stake native GEN tokens, but withdrawal, refund, and claim functions failed to deliver tokens back to caller wallets. Furthermore, contract deployers had no administrative recovery mechanism.

> **Root Cause Assessment:**
> - **GenVM Platform Issue (`genlayerlabs/genvm-manager#20`):** `emit_transfer(value=..., on="finalized")` records the outbound message in the transaction receipt, but validator execution nodes currently fail to dispatch native value on-chain.
> - **Smart Contract Architecture Gap:** Smart contracts lack owner/admin rescue mechanisms (`emergency_withdraw`). Furthermore, contract internal bookkeeping (e.g. `self.funded_amount = 0`) zeroed out balances immediately, permanently locking retry attempts.
> - **Unlinked Court & Escrow:** `NuanceDisputeCourt` rulings were not callable or enforceable by `NuanceEscrow` , permanently freezing disputed collateral.

This master build plan details the systematic engineering execution across four phases to remediate all fund-lock loopholes, connect the court to escrow settlements, optimize backend indexing, and resolve production deployment bottlenecks.

2. Phase 1: Smart Contract Upgrades & Redeployment

PHASE 1 • PRIORITY: IMMEDIATE

1.1 Emergency Withdrawal & Admin Escape Hatch

Each Intelligent Contract managing value must store an `admin: Address` in storage initialized to the deployer address (or a governance multi-sig) with an emergency escape hatch:

```
# contracts/nuance_escrow.py
@gl.public.write
def emergency_refund(self, recipient_override: str = "") -> None:
    if gl.message.sender_address != self.admin and gl.message.sender_address != self.creator:
        raise gl.vm.UserError("Unauthorized: Only admin or creator can invoke emergency refund.")
    if self.funded_amount <= 0:
        raise gl.vm.UserError("No funded amount to refund.")
```

```
    refund_target = Address(recipient_override) if recipient_override else self.creator
    amount = self.funded_amount
    self.funded_amount = 0
    self.status = "emergency_refunded"

    recipient = gl.get_contract_at(refund_target)
    recipient.emit_transfer(value=u256(amount), on="finalized")
```

1.2 Connecting Dispute Court to Escrow

Implement `execute_court_ruling` in `NuanceEscrow` to listen to verdicts produced by `NuanceDisputeCourt`:

- `court_address: Address` stored in `NuanceEscrow` constructor.
- Method `settle_dispute(dispute_id: u256)` queries `NuanceDisputeCourt.get_dispute(dispute_id)`.
- If `ruling == "favor_counterparty"`, releases milestone amount to counterparty. If `ruling == "favor_claimant"`, refunds the creator.

1.3 Prediction Market Cancellation & Unresolved Market Safety

Add `cancel_market(reason: str)` to `NuancePredictionMarket` allowing users to withdraw their original stakes (YES or NO) if the AI prompt fails, the source page is unreachable, or the market is cancelled.

1.4 Safe Payable Deposit Guard

Replace hard raises in `fund_escrow()` with a check before acceptance or ensure only designated senders can initiate payable methods.

3. Phase 2: Native Transfer Settlement & Workarounds

PHASE 2 • PRIORITY: HIGH

2.1 GenLayer Native Transfer Direct Verification

Mechanism	Status	Remediation / Strategy
`emit_transfer` to Contract	Experimental	Test recipient contracts with `__receive__` / `@gl.public.write.payable`
`emit_transfer` to EOA Wallet	Reported Bug #20	Engage GenLayer team; confirm if consensus finalization trigger is required
Dual-Ledger Accounting	Implemented	Continue tracking both `funded_amount` and real `contract_balance` via `eth_getBalance`

2.2 Fallback Claim Relayer & Faucet Rebalancing

To ensure live users on Bradbury testnet are never stranded when `emit_transfer` execution stalls:

- **Relayer Daemon:** Backend service listening for `FINALIZED` receipts of `claim_winnings`, `cancel_escrow`, and `retract_vote`.
- **Disbursement Verification:** If `contract_balance` remains unchanged after 60 seconds post-finalization, the relayer issues a direct native transfer from the treasury faucet wallet directly to the user, marking the transaction as `relayed_payout` in the database.

4. Phase 3: Backend & Architecture Hardening

PHASE 3 • PRIORITY: MEDIUM

3.1 Direct Python JSON-RPC Client (Eliminating Subprocesses)

Currently, `genlayer_indexer.py` executes `npx tsx scripts/genlayer-read.ts` as a Node child process every poll cycle (~3-15 seconds), consuming significant memory and CPU on Render.

- **Refactor:** Build native Python JSON-RPC caller using `httpx.AsyncClient` against `https://rpc-bradbury.genlayer.com`.
- **Impact:** Reduces memory footprint by ~180MB and CPU spikes by 90%, preventing Render free/standard tier OOM kills.

3.2 Database Engine Alignment (PostgreSQL & SQLite)

- Strictly maintain Alembic migrations across all 12 SQLAlchemy models.
- Ensure all datetime fields default to `DateTime(timezone=True)` across both SQLite local development and Render PostgreSQL.

5. Phase 4: Frontend & UX Polish

PHASE 4 • PRIORITY: MEDIUM

5.1 Environment Variable Synchronization

Update `.env.local` and Vercel project configuration to include all 6 contract addresses:

```
NEXT_PUBLIC_API_URL=https://nuance-backend-XXXX.onrender.com
NEXT_PUBLIC_ESCROW_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_DISPUTE_COURT_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_PREDICTION_MARKET_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_GOVERNANCE_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_VALIDATORS_CONTRACT_ADDRESS=0x...
NEXT_PUBLIC_AGENT_DIRECTORY_CONTRACT_ADDRESS=0x...
```

5.2 Actionable Resolution UX

- Replace passive *"funds stuck behind platform issue"* text with actionable status cards.
- Provide a one-click **"Request Faucet Rebalance"** or **"Retry On-Chain Claim"** button when the indexer verifies non-zero contract balance.

6. Master Implementation Roadmap & Milestones

Milestone	Deliverable	Est. Time	Verification Criteria
M1: Contract Patches	Patch `nuance_escrow.py`, `nuance_prediction_market.py` with emergency withdraw and dispute hook.	Day 1–2	Unit tests in GenVM simulator pass; all 6 contracts compile without TypeError.
M2: Redeployment	Execute `scripts/deploy.ts` against Bradbury testnet.	Day 2	All 6 contracts deployed; addresses synchronized in `backend/.env` and `.env.local`.
M3: Direct RPC Client	Implement `services/genlayer_native_rpc.py` in Python.	Day 3	Subprocess calls removed from indexer; zero TypeScript dependency during backend poll loop.
M4: Relayer Engine	Deploy claim relayer daemon for stuck native payouts.	Day 4	Users receive testnet GEN payout within 90s regardless of GenLayer protocol bug status.
M5: Full E2E Audit	Run Playwright E2E suite and complete live wallet connect on Vercel/Render.	Day 5	End-to-end escrow creation, milestone review, dispute ruling, and prediction claim verified live.

7. Production Release Checklist

- [ ] All contracts deployed to Bradbury testnet and addresses committed to repository.
- [ ] Render backend configured with `CORS_ORIGINS` matching production frontend domain and Vercel regex.
- [ ] Frontend Vercel deployment built with verified `NEXT_PUBLIC_API_URL`.
- [ ] Automated uptime ping active on `/health` to prevent Render free-tier cold sleep.
- [ ] All 315 backend unit tests passing in CI/CD pipeline.